Guia de Estudo — Rede Social

Explicação simples de cada arquivo, para todo o grupo

Sem pressa, sem jargão. Se travar em algum trecho, pode ler de novo — não tem problema.

Antes de tudo: o que é o sistema, em uma frase

É um caderninho de contatos que mora dentro de um arquivo no computador (o banco SQLite). O programa só sabe fazer seis coisas: cadastrar pessoa, apagar pessoa, ligar duas pessoas como amigas, desligar essa amizade, mostrar os amigos de alguém e mostrar todo mundo cadastrado.

Todo o resto do projeto — os sete arquivos .py — existe só para organizar essas seis ações em pedaços menores e mais fáceis de entender. Nenhum arquivo faz algo mágico ou complicado por si só; cada um é uma peça pequena de um quebra-cabeça simples.

A analogia que vale lembrar: um restaurante

Pense no sistema como um restaurante:

- main.py é o garçom — fala com o cliente (você, digitando no teclado), anota o pedido e leva a resposta de volta.
- rede.py é o balcão da cozinha — só repassa o pedido do garçom para quem realmente cozinha. Não cozinha nada sozinho.
- database.py é o cozinheiro — é quem realmente vai até a despensa (o banco de dados) e prepara o que foi pedido.
- init_db.py é quem monta a cozinha antes de abrir o restaurante — arruma as prateleiras (tabelas) onde os ingredientes (dados) vão ficar guardados.
- usuario.py é a etiqueta que vem em cada prato — não faz nada, só identifica bonitinho o que está sendo servido (os dados de um usuário).
- popular_banco.py é quem enche a despensa com alguns ingredientes de exemplo, só para já ter algo pronto para mostrar.
- teste_sistema.py é o inspetor sanitário — entra, testa um pouquinho de tudo, e diz se está tudo funcionando como deveria.

> **Por que isso importa:** Se alguém perguntar na apresentação "o que esse arquivo faz?", a resposta pode começar pela analogia do restaurante antes de entrar em código — isso já mostra que vocês entendem o papel de cada peça, mesmo sem saber decorar cada linha.

Gustavo — main.py, rede.py, revisão geral

Essas duas partes formam o garçom (main.py) e o balcão (rede.py) do restaurante.

rede.py — a parte mais simples de explicar

Esse arquivo é curto de propósito. Ele só tem uma classe chamada RedeSocial, e cada função dela faz uma única coisa: repassa o pedido para o database.py.

```
def adicionar_usuario(self, username, nome, idade):
    return database.adicionar_usuario(username, nome, idade)
```

Não existe lógica aqui — é tipo um garçom que não decide nada, só repete o pedido para a cozinha. Ele existe porque o professor pede modularização: main.py não deve falar direto com o banco de dados, então rede.py fica no meio dos dois.

main.py — o menu e as opções

É o arquivo mais longo do projeto, mas cada pedaço dele é simples. Ele fica em um loop (while True) mostrando o menu de novo e de novo, até a pessoa escolher "Sair".

```
while True:
    menu()
    opcao = input("Escolha uma opcao: ")
    match opcao:
        case "1": adicionar_usuario(rede)
        case "7": break
```

O match/case é só um jeito do Python escolher o que fazer dependendo do número digitado — é equivalente a um if/elif bem longo, só que mais organizado de ler. Cada opção do menu chama uma função (como adicionar_usuario, listar_perfis) que pede uma informação ao usuário (com input()) e manda para o rede.py cuidar disso.

Também vale mencionar a parte visual: as caixas desenhadas com caracteres como ╔═╗ não são nada complicado — são só textos especiais que, quando impressos em sequência, formam linhas e cantos. Isso foi adicionado para deixar a interface mais bonita e ganhar o ponto bônus do enunciado.

Sugestão para a apresentação do Gustavo

Conduzir a demonstração ao vivo do main.py é uma boa forma de mostrar o fluxo completo sem precisar explicar cada linha — a ação na tela já fala por si. Vale praticar a sequência: cadastrar um usuário, criar uma amizade, listar perfis, remover algo, e mostrar que tudo permanece salvo.

Pedro — database.py, init_db.py

Essas duas partes formam o cozinheiro (database.py) e quem monta a cozinha (init_db.py). É a parte tecnicamente mais densa do projeto, mas cada função resolve um problema bem concreto.

init_db.py — só cria as duas prateleiras

Esse arquivo roda uma vez e cria duas tabelas dentro do banco: usuarios (que guarda id, username, nome, idade) e amizades (que guarda dois números: usuario1 e usuario2, representando quem é amigo de quem).

```
CREATE TABLE IF NOT EXISTS usuarios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT UNIQUE NOT NULL,
    ...
)
```

O IF NOT EXISTS é o detalhe mais importante para explicar: ele diz "só crie essa tabela se ela ainda não existir". Por isso é seguro chamar essa função toda vez que o programa abre, sem apagar nada que já estava salvo.

database.py — onde tudo realmente acontece

Esse arquivo conversa direto com o banco. Vale focar a explicação em três funções, que são as que mais aparecem em perguntas de professor:

adicionar_usuario() tenta inserir o usuário, mas se o username já existir, o SQLite recusa automaticamente (por causa do UNIQUE lá na tabela) e gera um erro chamado

IntegrityError. O código captura esse erro com um try/except e devolve um simples "não deu" (False), em vez de travar o programa.

criar_amizade() é a função mais cuidadosa do projeto. Antes de inserir, ela checa três coisas: se a pessoa está tentando ser amiga de si mesma, se algum dos dois usernames não existe, e se a amizade já existe (verificando os dois lados possíveis, porque amizade entre joão e maria é a mesma coisa que entre maria e joão).

remover_usuario() faz duas etapas na ordem certa: primeiro apaga as amizades dessa pessoa, e só depois apaga a pessoa em si. Se fosse na ordem contrária, ficariam amizades "penduradas" apontando para alguém que não existe mais no banco.

Sugestão para a apresentação do Pedro

Não é necessário explicar todas as funções de database.py com o mesmo nível de detalhe. As três acima (cadastro com erro tratado, criação de amizade com as três checagens, remoção em ordem correta) já cobrem o que normalmente é perguntado e demonstram domínio técnico suficiente.

Vinícius — usuario.py, popular_banco.py, teste_sistema.py, README

Essa é a parte mais tranquila tecnicamente, mesmo parecendo bastante coisa por estar dividida em quatro itens. Vamos destrinchar cada um devagar.

usuario.py — a etiqueta do prato

Esse é, sem exagero, o arquivo mais simples do projeto inteiro. Ele só guarda quatro informações sobre uma pessoa: id, username, nome e idade.

```
class Usuario:
    def __init__(self, user_id, username, nome, idade):
        self.id = user_id
        self.username = username
        self.nome = nome
        self.idade = idade
```

Uma classe em Python é como uma ficha de cadastro em branco: define quais campos existem, mas cada vez que alguém usa Usuario(...), está preenchendo uma ficha nova com dados diferentes. self é só uma forma de dizer "esses dados pertencem a essa ficha específica", não a outra.

Se alguém perguntar "por que isso existe?": é mais fácil escrever usuario.nome do que ficar lembrando que o nome está na posição 2 de uma lista de números. A classe dá nome às coisas.

popular_banco.py — encher a despensa

Esse script existe só para não precisar digitar usuário por usuário manualmente toda vez que for testar ou apresentar o sistema. Ele faz duas coisas: limpa o banco (apaga tudo que tinha antes) e depois insere uma lista pronta de usuários e amizades de exemplo.

```
usuarios = [
    ("joao123", "João Silva", 20),
    ("maria456", "Maria Souza", 19),
    ...,
]
for username, nome, idade in usuarios:
```

```
    rede.adicionar_usuario(username, nome, idade)
```

O for aqui é só um jeito de repetir a mesma ação (cadastrar) para cada item da lista, sem copiar e colar a linha cinco vezes. A limpeza do banco antes é importante porque, sem ela, rodar o script duas vezes tentaria cadastrar o mesmo username de novo, e isso falharia silenciosamente.

> **Dica para a apresentação:** Rodar python popular_banco.py ao vivo e mostrar a mensagem "Banco populado com sucesso!" já é suficiente — não é necessário explicar linha por linha, só dizer o que o script faz e por que ele é útil (gerar dados rápido para demonstração).

teste_sistema.py — o inspetor

Esse arquivo só verifica, de forma automática, se as coisas mais importantes do sistema continuam funcionando. Ele faz uma sequência de ações (cadastrar, tentar duplicar, criar amizade, etc.) e, depois de cada uma, confere se o resultado foi o esperado.

```
resultado = rede.adicionar_usuario(usuario_teste, "Usuario Teste", 20)
checar("Cadastro de usuario", resultado)
```

checar() só compara: "o resultado que eu esperava bateu com o que realmente aconteceu?" Se sim, marca certo (✓); se não, marca errado (✗). No final, soma quantos testes passaram e quantos falharam.

Não é necessário entender cada teste individualmente para apresentar isso — basta rodar python teste_sistema.py ao vivo e mostrar a lista de certos no final, explicando que cada linha confirma que uma funcionalidade do sistema continua funcionando.

README.md — o manual do projeto

É só um arquivo de texto que explica, para qualquer pessoa de fora, o que o projeto faz e como rodá-lo. Não tem lógica nem código de verdade — só a documentação. Pode ser mostrado rapidamente na apresentação como prova de que o projeto está bem documentado, sem precisar ler tudo em voz alta.

Sugestão para a apresentação do Vinícius

Como combinado no grupo, a ideia é não gastar muito tempo com README e teste_sistema.py durante a apresentação — uma menção rápida de cada já é suficiente. O foco maior pode ficar em explicar a classe Usuario (que é simples e rápida) e rodar o popular_banco.py ao vivo, que é uma ação visual e fácil de acompanhar.

> **Se faltar tempo de estudo:** As partes mais fáceis de garantir com pouco tempo são a classe Usuario (quatro campos, sem lógica) e a explicação do popular_banco.py ("é só uma lista de exemplos que insere usuário por usuário"). Isso já cobre uma boa parte da nota dessa seção sem precisar memorizar muito.

Se o tempo apertar: o que priorizar

Com pouco tempo de estudo, vale focar nestas perguntas, que são as mais prováveis de aparecer e cobrem o essencial de cada parte:

- O que esse arquivo faz, em uma frase? (todos devem saber responder isso sobre seu próprio arquivo)
- Por que a amizade verifica as duas ordens possíveis? (resposta: porque amizade vale nos dois sentidos)
- Por que remover usuário apaga as amizades primeiro? (resposta: para não deixar amizade "perdida" apontando para alguém que não existe mais)

- Por que não tem login? (resposta: não foi pedido pelo professor, e aumentaria a complexidade sem necessidade)

Se cada integrante conseguir responder com segurança essas quatro perguntas sobre sua própria parte, a apresentação já fica bem cobrida, mesmo sem saber de cor cada linha de código.

Um lembrete final

Ninguém precisa saber tudo sobre o projeto inteiro — cada um precisa estar confortável com a sua parte, e ter uma ideia geral de como ela se conecta com o resto. Se em algum momento da apresentação faltar uma resposta, está tudo bem outro integrante complementar — isso é normal em trabalho de grupo e não pesa contra vocês.